Suggested Process for Data Analysis

Refined with DeepSeek

Concepts

The research paper you attached (Kang, 2022) directly addresses this. The author uses the **Google Borg trace v3** and has already solved the exact aggregation problem you're facing. Let me break down how they did it, based on the paper.

The Core Answer: The "Collection" is the Job

In the Borg trace, a user-facing "job" is not a single row in a log file. Instead, it is represented as a `collection` (specifically, a `collection` of type `job`).

The paper explicitly defines this in **Section 3.1 (Data Table)** , point #3:

> **Collection events:** The table represents the life cycle of **collection instances (jobs or alloc sets)**… The table also describes fields specific to the property for collection instance, such as collection type (**a job** or alloc set)…

So, to aggregate information to the job level, you must use the `collection_events` table as your master reference for jobs.

Step-by-Step Aggregation Strategy (Based on the Paper)

The paper's **Section 3.2 (Data Process Pipeline)** and **Figure 3.1** outline the exact process. Here's how you would implement it:

Step 1: Identify Your Jobs Query the `collection_events` table. Filter for records where the `collection_type` indicates a `job`. Each unique job will have a `collection_id` (or similar unique identifier, like `job_id` in the actual schema). This table gives you the job's *requested* resources (CPU, RAM), its priority/tier, and its lifetime (start and end times).

Step 2: Find the Tasks Belonging to Each Job The paper states that a job consists of multiple tasks. This relationship is captured in the `instance_events` table.

> **Instance events:** The table provides actions and life cycle information for **instances (tasks** or allocs). The table indicates **resource request**… (Section 3.1, point #4)

The `instance_events` table will have a foreign key (likely `collection_id`) linking each task/instance back to its parent job.

Step 3: Get the Actual Runtime Usage for Each Task This is the most important step for prediction. The paper uses the `instance_usage` table.

> **Instance usage:** The table records the **usage statistics of each task** in approximately 5-minutes windows… (Section 3.1, point #5)

This table holds the *actual* consumed CPU and memory (not just the requested amounts). It is linked to `instance_events` via `instance_id` (and implicitly `collection_id`).

Step 4: Perform the Aggregation (The "Merge") The paper describes this precisely (Section 3.2):

> "…designated queries are run on Big Query to **merge collection events, instance events, and instance usage tables on each job identifier**… to obtain information for **collection properties, instance requested information, and usage data recorded at run-time**."

In SQL (on Google BigQuery), this looks like a series of `JOIN` operations:

```sql
-- Pseudo-code for aggregation
SELECT
    c.collection_id,
    c.priority,  -- Job-level priority (tier)
    c.resource_request_cpu, -- Job's requested CPU
    AVG(u.actual_usage_cpu) AS avg_task_cpu_usage, -- Aggregated usage
    COUNT(i.instance_id) AS num_tasks_in_job,
    MAX(i.end_time) - MIN(i.start_time) AS job_lifetime

FROM `collection_events` c

-- Join to get all tasks (instances) for this job
LEFT JOIN `instance_events` i
    ON c.collection_id = i.collection_id

-- Join to get the actual usage records for each task
LEFT JOIN `instance_usage` u
    ON i.instance_id = u.instance_id
    AND i.collection_id = u.collection_id

GROUP BY c.collection_id, c.priority, c.resource_request_cpu
```

How to Model "Job Lifetime" for Prediction

The paper uses "total lifetime of jobs" as a feature in their **Lasso Regression** model (Section 5.2, Table 5.1). Here's how you can derive this:

- **Job Lifetime** = `MAX(end_time) - MIN(start_time)` for all instances belonging to that job.
- You can get `start_time` and `end_time` from the `instance_events` table (look for `EVENT_TYPE` like `EVENT_TYPE_START` and `EVENT_TYPE_STOP` or `EVENT_TYPE_FINISH`).

For a **prediction model**, you would structure your data like this:

Feature (X variables)	Target (y variable)
Job Priority (Tier)	Job Lifetime
Requested CPU / RAM	
Number of Tasks in Job	
Historical avg. task usage (from past similar jobs)	
Scheduling Class (latency-sensitive or not)	

Summary of the Paper's Approach

Your Goal	How the Borg Trace Represents It	Table to Use
Find all Jobs	Each `collection` of type `job`	`collection_events`
Find Tasks belonging to a Job	Each `instance` is a task, linked by `collection_id`	`instance_events`
Get actual resource usage for a Task	Time-series records per `instance_id`	`instance_usage`
Get Job Lifetime	Time difference between first start and last end of its `instances`	Calculate from `instance_events`

Important Caveats (from the paper)

1. **Sample First:** The paper notes that the full trace is ~2.4 TiB. They used a **1% uniform random sample** (Section 3.2) to work locally. You should do the same.
2. **Anonymization:** All text fields are hashed, and numeric resources are linearly transformed. You cannot know the absolute values (e.g., "this is 4 CPU cores"), but you can compare *relative* values.
3. **"Churn" Phenomenon:** The paper mentions (Section 2.3) that many tasks are resubmitted. Your job lifetime calculation should account for this, perhaps by tracking a job across resubmissions using its persistent `collection_id`.

By following this aggregation strategy—starting with `collection_events`, linking to `instance_events`, and finally joining to `instance_usage`—you can reconstruct the full, multi-level hierarchy of jobs and their tasks, which is exactly what the paper did for its analysis.

==========

Additional Resources

https://www.researchgate.net/publication/372951052_A_Deep_Dive_into_the_Google_Cluster_Workload_

https://colab.research.google.com/github/google/cluster-data/blob/master/clusterdata_analysis_colab.ipynb

https://oaktrust.library.tamu.edu/server/api/core/bitstreams/9794dea4-371a-490b-8503-6f0ba041364f/content

“““ 2.1 Borg Units 2.1.1 Tasks & Collections A job in Borg consists of one or more tasks that all run in the same program, defined by some general properties (e.g., name, owner, number of tasks in a job) and constraints that force particular tasks to run on specific attributes [14]. To avoid extra overhead in virtualization, Borg maps each task directly to a set of Linux processes running in a container on a machine so that the majority of the workload does not need to run inside virtual machines (VMs) [18]. A Borg alloc (short for allocation) reserves resources for one or more tasks on a machine in the future and can gather similar tasks from different jobs onto the same machine to optimize efficiency. A group of alloc instances forms an alloc set that reserves resources on multiple machines into which users can schedule additional jobs [14]. “““

https://static.googleusercontent.com/media/research.google.com/en//pubs/archive/43438.pdf [explains google cloud data]

https://www2.eecs.berkeley.edu/Pubs/TechRpts/2010/EECS-2010-95.pdf

https://github.com/Vu5e/JobFailurePredictionGoogleTraces2019

==========

Structured Breakdown

Here is the full, corrected summary.

Part 5: Final Summary – What You Should Do

Based on all the research papers, here is your validated methodology with your specific constraints.

Step 1: Scope Definition – Filter Before You Download

Your specifications: - Focus on **batch jobs only** (jobs that do not run indefinitely) - Filter out **production jobs** (long-running services) - Limit to **one cluster** (cell a – one geographical region) - Limit to a **limited number of tenants** (subscriptions/users)

Paper support from Borg paper (Verma et al., 2015), Section 2.5:

> *"Production tier (priorities 120–359) are the highest priorities in normal use… Batch jobs are much less sensitive to short-term performance fluctuations."*

Priority ranges from Google trace documentation v3:

Tier	Priority Range	Your Action
Production	120–359	**FILTER OUT** (long-running, latency-sensitive)
Monitoring	360	**FILTER OUT** (infrastructure)
Mid-tier	116–119	**KEEP** (batch-focused)
Best-effort Batch (beb)	100–115	**KEEP** (batch jobs)
Free tier	99	**KEEP** (batch/test jobs)

From **Kang (2022), Section 3.2:**

> *"We apply a **uniform random sampling at 1%** , and the sample is fetched for additional data processing."*

Your action plan: 1. Filter by `priority` 119 (exclude production and monitoring) 2. Filter by specific `user` (tenant) IDs of interest 3. Filter by specific `cell` (e.g., cluster `a`) 4. Then apply 1% sampling if still too large

---

Step 2: Reduce Time Granularity

Your specification: - Reduce granularity from 5 minutes to a larger window (1 hour, 4 hours, or 1 day) - Keep granularity at the time level, aggregated per job

Your action plan:

Original Granularity	Your Resampled Granularity	Reduction Factor
5 minutes	1 hour	12× reduction
5 minutes	4 hours	48× reduction

Original Granularity	Your Resampled Granularity	Reduction Factor
5 minutes	1 day	288× reduction

From **Zhi (2025), Section 5.9.2 "Time window data":**

> *"When the time window shifts beyond T, the algorithm removes the data from the beginning of the window and inserts new data at the end."*

Step 3: Identify Jobs – Task to Job Aggregation

Each unique `JobID` = one job.

From **Chen et al. (2010), Section 3.1:**

> *"Each row in the dataset represents the execution of a single task… Each task belongs to a single job."*

Your action: - Group all records by `JobID` - A job may have one task or many tasks - All tasks belonging to the same job share the same `JobID`

Step 4: Calculate Job Lifetime

From **Chen et al. (2010), Section 4.1:**

> *"Job duration is calculated as the time difference between first task start and last task end."*

Formula: > Job Lifetime = MAX(task_end_time across all tasks in job) − MIN(task_start_time across all tasks in job)

Note: For batch jobs (your focus), this is a finite value.

Step 5: Calculate Job Resource Usage at Each Time Interval (CRITICAL – CPU vs Memory)

The Core Rule:

Resource	Behavior	Why	What we calculate
CPU	Compressible. Tasks share.	CPU takes turns.	**MAX** of all tasks running at that time

Resource	Behavior	Why	What we calculate
Memory	Non-compressible. Cannot share.	Each task needs its own memory.	**SUM** of all tasks running at that time

What "overlap" means: Tasks are running at the same time.

From **Borg paper (Verma et al., 2015), Section 6.2:**

> *"If a machine runs out of non-compressible resources (memory), the Borglet immediately terminates tasks."*

From **Borg paper, Section 5.5:**

> *"Tasks are permitted to consume resources up to their limit. Most of them are allowed to go beyond that for compressible resources like CPU."*

Example – At the same moment in time:

Task	CPU	Memory
Task A	0.5	2GB
Task B	0.3	3GB
Task C	0.4	1GB

What the job needs at that moment: - **CPU:** MAX(0.5, 0.3, 0.4) = **0.5** (they share) - **Memory:** SUM(2GB, 3GB, 1GB) = **6GB** (they cannot share)

For each time window (1 hour, 4 hours, or 1 day):

What we calculate	How	Why
Max overlapped CPU usage	Look at all moments in the window. Take the highest single MAX value across all moments.	This tells you the peak CPU demand when the most tasks were competing.
Max total Memory usage	Look at all moments in the window. For each moment, add up memory of all tasks. Take the highest SUM across all moments.	This tells you the peak memory demand when the job needed the most RAM.

Keep the time series per job:

Job has (CPU_t, MEM_t) for t = 1, 2, 3, ..., T where: - `CPU_t` = MAX CPU usage among all tasks in job running during window t - `MEM_t` = SUM of memory usage of all tasks in job running during window t - `T` = job lifetime in resampled windows - `t` = granularity time: per hour, per 4 hours - this is arbitrary and selected by the data analysis and has pros and cons

From **Resource Central paper (Cortez et al., 2017), Section 6.2:**

> *"We aggregate utilization data in the simulator by **adding up the co-located VMs' maximum utilizations in each 5-minute period**."*

Example of the full time series (your own, corrected):

Time Window	Task A CPU	Task B CPU	Job CPU (MAX)	Task A MEM	Task B MEM	Job MEM (SUM)
Hour 1	0.5	0.25	**0.5**	2GB	3GB	**5GB**
Hour 2	0.5	0.25	**0.5**	2GB	3GB	**5GB**
Hour 3	0.0	0.25	**0.25**	0GB	3GB	**3GB**

Job CPU time series = [0.5, 0.5, 0.25] Job MEM time series = [5GB, 5GB, 3GB]

Part 6: Complete Workflow Diagram

```
Raw Google Borg Trace (1 month, cell a)


  STEP 1: FILTER
  - priority  119 (batch/free tiers)
  - specific tenant IDs
  - cell = 'a'
  - 1% sampling (if needed)



  STEP 2: RESAMPLE TIME
  - 5 minutes → 1 hour (or 4h / 1 day)
  - Keep time series structure
```

```
STEP 3: AGGREGATE BY JOB
- Group by JobID
- Job Lifetime = MAX(end) - MIN(start)
```

```
STEP 4: TIME-SERIES PER JOB
For each time window t:
  CPU_job(t) = MAX(task CPU usage)
  MEM_job(t) = SUM(task MEM usage)
  (only for tasks running at the same time)
```

```
STEP 5: EXTRACT FEATURES
- Job Lifetime
- CPU time series → mean, P90, P95
- MEM time series → mean, P90, P95, max
```

```
OUTPUT: Job-level dataset
One row per job with time-series
features for prediction
```

Part 7: Critical Distinctions to Remember

Concept	Definition	Source
Request / Limit	Tenant's stated need	`resource_request` field
Reservation	Borg's prediction of actual need	Calculated by Borg
Usage	Actual measured consumption	`average_usage` / `maximum_usage`
Job	Logical application unit	Has `JobID`
Task	Container running a job	Has `TaskID`, belongs to one `JobID`
Process	Inside a container	Not traced

Concept	Definition	Source
Batch Job	Finite duration, less latency-sensitive	Priority 100–119 (beb, mid)
Production Job	Long-running, latency-sensitive	Priority 120–359 (filter these out)
CPU Aggregation	MAX of concurrent tasks (compressible, tasks share)	Borg paper Section 6.2
Memory Aggregation	SUM of concurrent tasks (non-compressible, cannot share)	Borg paper Section 6.2
Overlap	Tasks running at the same time	Defined in this summary

---

Part 8: Data Size Estimation

From **Kang (2022), Section 3.2:**

> *"The Google Borg trace V3 is about **2.4 TiB compressed**."*

Your reductions:

Filter	Reduction Factor
Cell only (one of 8 cells)	~8×
Batch jobs only (priorities 119)	Significant (production jobs dominate)
Limited tenants	Variable
1% sampling	100×
Resample 5 min → 1 hour	12×

From **Chen et al. (2010), Section 3.1:**

> *"The trace contains 75 five-minute reporting intervals. There are a total of **3,535,029 observations, 9,218 unique jobs**."*

After aggregation, 9,218 rows is tiny. Your 2019 v3 dataset will be larger but still manageable after filtering.

Part 9: Final Validation Checklist

Your Requirement	Paper Support	Status
Batch jobs only	Borg paper Section 2.1, priority ranges	
Filter production jobs	Borg paper Section 2.5	
One cluster (cell a)	Google trace documentation v3	
1% sampling	Kang (2022) Section 3.2	
Resample to larger time windows	Zhi (2025) Section 5.9.2	
CPU = MAX of concurrent tasks	Borg paper Section 6.2 (compressible)	
Memory = SUM of concurrent tasks	Borg paper Section 6.2 (non-compressible), Cortez (2017) Section 6.2	
Keep time series, not just one number	Quasar paper Section 2.2	
Percentiles of time series	Zhi (2025) Section 2.8.2	
Job lifetime formula	Chen et al. (2010) Section 4.1	

Part 10: Simple Summary

For each time window: - **CPU = highest single task** (because tasks share) - **Memory = sum of all tasks** (because they cannot share) - Take the **maximum** across all moments in that window for both - Keep the time series. Do not collapse to one number.

Filter to batch jobs only (priority 119), one cell, limited tenants. Resample 5 minutes to 1 hour (or longer). Sample 1% if still too large.